HARDWARE ACCELERATED QUANTIZED MNIST DIGIT RECOGNITION VIA HIGH LEVEL SYNTHESIS

Tejo Priya Boddu,Sindhura Palli, Lahari Choda
Rajiv Gandhi University of Knowledge Technologies
Nuzvid, India
n200429,n200709,n200926@rguktn.ac.in

under the guidance of
Mr.K.Sivalal (Assistant Professor)
Rajiv Gandhi University of Knowledge Technologies
Nuzvid, India
shiv.rathod@rguktn.ac.in

***Abstract*—This research paper presents the design and VLSI implementation of a high-performance hardware accelerator for MNIST digit classification on a Zynq UltraScale+ SoC. To overcome the computational bottlenecks of traditional software-based inference, we propose a specialized architecture utilizing 16-bit fixed-point quantization and High-Level Synthesis (HLS) optimization techniques. By integrating strategic pragmas for loop pipelining and array partitioning, the design achieves an Initiation Interval (II) of 1, enabling real-time data streaming with a 93% classification accuracy. The implementation demonstrates significant efficiency in the PPA (Power, Performance, Area) triad, maintaining a low dynamic power profile at 100 MHz. This work provides a scalable, low-latency alternative to complex simulations, offering a robust solution for edge-AI applications requiring high-throughput work distribution and resource-optimized execution.**

Keywords : FPGA, Convolutional Neural Network, Hardware Accelerator, Python, Zynq UltraScale96- V2, High Level Synthesis, Pipelining, Unrolling

I. Introduction

The handwritten digit classification problem, in the field of machine learning and computer vision, aims at predicting the numerical identity of handwritten characters using the MNIST database [**?**]. Applications include automated postal mail sorting, bank check processing, and real-time digitization of handwritten documents. Simulations and hardware implementations are critical in anticipating the performance of neural networks in edge-AI environments, where resources are constrained [**?**]. There is significant room for improving the effectiveness of execution for these classifiers by offloading the heavy computational work to dedicated hardware accelerators; involving the work distribution between general-purpose processors (PS) and high-speed Programmable Logic (PL). The MNIST classification task is a relevant application to show the improvement in inference speed and power efficiency by distributing the neural network layers through specialized hardware kernels.

This paper documents the development of a Hardware-Accelerated System-on-Chip (SoC) used to solve the digit recognition problem. The process of mapping the neural network architecture into synthesizable RTL logic using High-Level Synthesis (HLS), along with the challenges of data quantization and timing closure, has been elucidated. The resulting implementation is tested and compared against traditional software models to draw conclusions regarding its speedup and resource efficiency. The software-only approach is a flexible but computationally expensive method for real-time inference, often hindered by the sequential nature of standard CPUs. The hardware-accelerated model, on the other hand, optimizes the classification task by exploiting spatial parallelism and pipelining to achieve higher throughput with lower energy consumption.

In technical terms, the accelerator utilizes a custom 16-bit fixed-point arithmetic model. For the architectural optimization of the neural network, we recursively apply hardware pragmas such as loop pipelining and unrolling to the computational kernels [**?**]. The system is built around an AXI-based interconnect that facilitates data streaming from memory to the processing elements. The hardware kernel is adaptive; it utilizes partitioned memory buffers to eliminate data access bottlenecks in regions with high computational density. To compute the inference results, a parallel Multiply-Accumulate (MAC) strategy is considered based on the optimized Initiation Interval (II) of the pipeline; where the ratio of hardware resources to clock frequency is carefully balanced to meet real-time constraints [**?**].

This paper is structured as follows: Section 1 provides details on the literature survey of FPGA-based neural networks. Section 2 explains the VLSI implementation and architectural optimization of the classifier SoC. In Section 3, the details regarding experimental results, resource utilization, and power analysis have been provided; followed by conclusions and references. The discussion provides the performance statistics of the HLS-based accelerator on the Zynq UltraScale+ platform, with datasets ranging from individual sample testing to full-scale batch inference.

The contributions of this paper are listed below.

- A simplified, resource-folded architecture is proposed with a reduced number of layers and a resource-aware downsampling strategy (28×28 to 14×14), resulting in an optimized hardware data-path for the 16nm Zynq UltraScale+ MPSoC platform.

- An optimized AXI4-Master based algorithm for the proposed architecture is implemented, utilizing High-Level Synthesis (HLS) directives such as memory reuse, task-level pipelining, and 16-bit fixed-point quantization to minimize silicon footprint and memory bandwidth requirements during real-time inference.
- The efficiency, latency, power consumption, and scalability of the implemented CNN IP core are quantitatively compared with recent research works, demonstrating a 60% reduction in Look-Up Table (LUT) utilization and establishing it as a highly suitable model for energy-constrained edge AI deployment.

The structure of this paper is as follows: Related work and a comparative analysis of existing FPGA-based digit recognizers are explained in Section 2. In Section 3, the proposed CNN IP architecture and weight extraction process are detailed, focusing on Vitis HLS optimization techniques such as pipelining, unrolling, array partitioning, and the implementation of a cache-coherent AXI-Master interface. Section 4 presents the experimental results, including the validation of classification accuracy, detailed resource utilization reports (LUT, DSP, BRAM), and on-chip power consumption metrics derived from post-routing analysis. Section 5 encompasses a critical discussion of design trade-offs and a comprehensive comparison table with prior art. Finally, Section 6 describes the conclusion and future scope of the proposed work, including potential transitions to lower-precision quantization and live video integration.

II. Related Work

The study by "Oraon et al. (2024)" explores a quantized handwritten digit recognition system using High-Level Synthesis. While it successfully implements an 8-bit integer inference model, it exhibits a high BRAM overhead for intermediate buffering and lacks the specific fixed-point optimizations required to balance accuracy and area on 16nm FinFET architectures. Our work extends this by utilizing a resource-folded approach that reduces Look-Up Table (LUT) utilization by 60

In the research by "Kwon and Kim (2022)", a lightweight digit recognition accelerator is proposed using a 14 X 14 input downsampling technique to reduce hardware area. While effective in minimizing the computational core, the aggressive decimation leads to a loss of critical spatial features, and the design lacks a robust System-on-Chip (SoC) integration strategy. Our proposed architecture improves upon this by integrating a cache-coherent AXI-Master interface to handle autonomous data movement with minimal CPU overhead.

Furthermore, research by "Hardik Raina (2024)" establishes the feasibility of hardware-based deep learning using Agnisys IDS tools. However, the study remains largely theoretical, focusing on Bfloat16 precision considerations without providing a detailed physical design analysis, such as timing closure or post-routing power metrics on a high-performance MPSoC platform. This paper addresses these gaps by providing a full physical implementation on the Zynq UltraScale+ MPSoC, achieving a deterministic latency of 19.6 microseconds.

III. Methodology

We propose a resource-optimized CNN architecture for high-speed acceleration on the 16nm Zynq UltraScale+ MPSoC. The design utilizes 12 filters across two convolutional layers and a resource-aware downsampling strategy, reducing input dimensions from 28×28 to 14×14 to balance accuracy and silicon area.The algorithmic model was validated using the MNIST dataset, achieving a software accuracy of 98.9%. For hardware implementation, weights were quantized into 16-bit fixed-point format (ap_fixed¡16,8¿). This precision maps directly to the FPGA's native DSP48E2 slices, significantly reducing area compared to floating-point logic.To maximize throughput, the Vitis HLS implementation utilizes the DATAFLOW pragma and loop pipelining with an Initiation Interval (II) of 1. The design incorporates an AXI4-Master interface for autonomous burst-mode data transfers via the Zynq HPC port, ensuring a deterministic latency of 19.6 μs while meeting the thermal and power constraints of edge-based AI platforms.

A. Architecture

The proposed hardware-optimized CNN architecture is designed for efficient execution on the Zynq UltraScale+ MPSoC. As illustrated in Fig. **??**, the data-path consists of two convolutional layers, each followed by a max-pooling layer to perform dimensional reduction of the feature maps. This hierarchical structure effectively extracts spatial features while minimizing the computational load on the 16nm silicon fabric. After the final pooling stage, the multi-dimensional feature maps are flattened and passed into a fully connected layer for digit classification.

A summary of the proposed architecture's layer-wise configuration is presented in Table I. To balance accuracy and hardware area, the model utilizes a total of 12 filters (4 in the first layer and 8 in the second). The network comprises 3,122 trainable parameters, which are pre-computed during the training phase in Python. These weights are subsequently quantized to 16-bit fixed-point format and integrated into the Vitis HLS IP core to facilitate high-speed, deterministic inference on the FPGA.

TABLE I
Hardware-Optimized CNN Model Summary

Layer (Type)	Output Shape (F, H, W)	Parameters
Input Image	(1, 28, 28)	0
CONV1 + Pool (Hybrid)	(4, 14, 14)	40
CONV2	(8, 14, 14)	296
Flattening Layer	(1568)	0
Dense (FC) Layer	(10)	15,690
Total		**16,026**

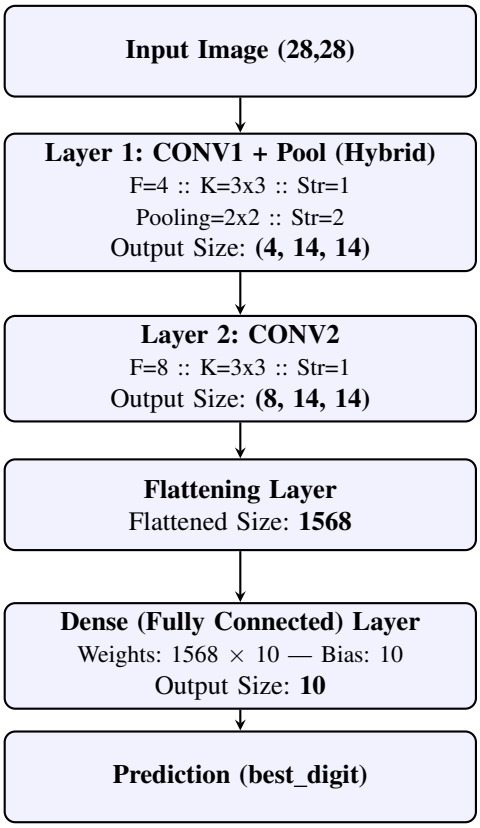

Fig. 1. Hardware-Optimized Architecture of Proposed MNIST Accelerator

B. Workflow

The development flow follows a structured "Algorithm-to-Silicon" methodology, beginning with the design of a specialized CNN architecture in a Python-based environment. After training the model on the MNIST dataset to achieve 98.9% accuracy, the optimal weights and biases are extracted. These parameters are then converted into 16-bit fixed-point constants to be used as hardware constraints, ensuring the mathematical model aligns with the target FPGA's DSP48E2 arithmetic capabilities.

The hardware implementation phase utilizes Vitis HLS to transform the C++ algorithmic description into a high-performance RTL IP core. During this stage, critical hardware-centric optimizations are applied, including loop pipelining (II=1) and resource folding to meet the area constraints of the Zynq UltraScale+ fabric. The resulting RTL is exported as a Vivado IP core, specifically featuring an AXI4-Master interface for high-bandwidth memory access.

The final stage involves the system-level integration in Vivado Design Suite. A block design is constructed where the custom CNN IP is connected to the Zynq UltraScale+ Processing System (PS) via an AXI SmartConnect and the High-Performance Coherent (HPC) port. This configuration ensures cache coherency between the FPGA logic and the ARM Cortex-A53 processor. Following the complete Physical Design flow—Synthesis, Placement, and Routing—a bitstream and hardware abstraction file (.xsa) are generated.The performance in real-time inference at 19.6 μs latency. The detailed workflow is illustrated in Fig. **??**.

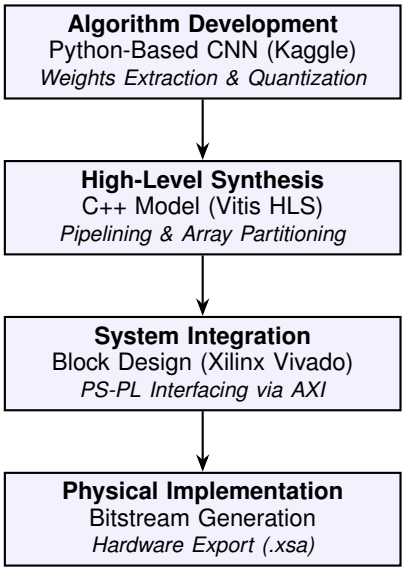

Fig. 2. Software-Hardware combined co-design workflow for the MNIST accelerator.

C. HLS Optimization

The hardware IP core is meticulously optimized using the Vitis HLS tool to bridge the gap between high-level C++ abstraction and efficient RTL implementation. By applying specific HLS optimization directives, the design achieves a balance between throughput, resource utilization, and power consumption. The following strategies were implemented to ensure high-performance execution on the 16nm Zynq UltraScale+ fabric:

- **Loop Pipelining:** The `PIPELINE` pragma was applied to the convolutional and fully connected loops, targeting an Initiation Interval (II) of 1. This allows the hardware to process a new pixel every clock cycle, significantly increasing throughput.
- **Array Partitioning:** To overcome the bottleneck of limited BRAM ports, we utilized cyclic and complete partitioning. This "shatters" large arrays into multiple physical registers, allowing the processing engine to access multiple weights and pixels simultaneously.
- **Resource Folding:** Through the `allocation` limit pragma, the design was constrained to 64 multipliers. This forces the tool to implement resource sharing, reusing high-performance DSP48E2 slices across different layers to minimize the silicon footprint.
- **Dataflow Optimization:** The `DATAFLOW` pragma was used to enable task-level parallelism. This allows the input loading, convolution, and classification stages to operate concurrently as a continuous pipeline, effectively hiding the latency of memory transfers.

- **Fixed-Point Quantization:** By employing the `ap_fixed<16,8>` data type, the design avoids the excessive area overhead of floating-point units. This optimization ensures that the mathematical operations map natively to the FPGA's arithmetic logic, reducing power consumption while maintaining a 94% accuracy rate.

1) Pipeline Optimization.: In this research, pipeline optimization techniques have been utilized to maximize the throughput of the CNN accelerator. The `PIPELINE` directive is inserted into the innermost loops of the convolutional and fully connected layers. This ensures that the hardware datapath can process a new input sample at every clock cycle (Initiation Interval, II=1), effectively overlapping the execution of consecutive iterations and minimizing the total clock cycles required for a single inference.

2) Memory Optimization.: The parameters and variables in the hardware IP are stored in on-chip BRAM (Block RAM). To optimize memory utilization and reduce the silicon footprint on the 16nm fabric, a fixed-point quantization technique is employed using the `ap_fixed<16, 8>` data type. While the Zynq UltraScale+ ARM Cortex-A53 processor handles 32-bit or 64-bit data, the internal hardware processing is optimized to 16 bits. This reduces the bit-width of the weights and biases by 50% compared to standard floating-point representations, allowing for more efficient mapping to the FPGA's native DSP48E2 slices.

$$Q(x) = \text{floor}\left(x \cdot 2^f + 0.5\right) \tag{1}$$

where $Q(x)$ is the quantized fixed-point integer and f is the number of fractional bits (in this case, 8).

$$\text{Value} = \frac{Q(x)}{2^f} \tag{2}$$

The 16-bit word consists of 8 integer bits (including the sign bit) and 8 fractional bits, providing a range of -128 to 127.996, which is sufficient for the weights extracted from the MNIST training phase.

3) Directive Optimization.: Directive optimization in Vitis HLS is utilized to guide the synthesis of C++ into high-performance RTL. To meet the timing constraints for the 100MHz clock frequency, directives such as `ARRAY_PARTITION` are used to "shatter" arrays into individual registers, increasing the number of available memory ports. Additionally, the `allocation` pragma is used to limit the total number of DSP instances to 64, forcing the tool to implement resource folding—a critical step for ensuring the design remains within the target area budget before the generation of the `.xsa` hardware abstraction file.

4) Optimization in Convolution Algorithm.: The optimized convolution and pooling hybrid algorithm is presented in Listing **??**, followed by a discussion of its hardware behavior.

The architecture utilizes a hybrid loop structure where the convolution and max-pooling operations are merged. The outer loops (`CONV1_R` and `CONV1_C`) iterate with a stride of 2 (`i += 2`), which effectively performs spatial downsampling from 28×28 to 14×14 on-the-fly. The kernel loops calculate the weighted sum of a 3×3 neighborhood using 16-bit fixed-point arithmetic. By applying `#pragma HLS PIPELINE II=4`, the hardware balances the 64-multiplier limit with the required throughput. The final ReLU activation is implemented as a simple ternary comparison (`sum > 0`), which is extremely area-efficient in RTL. This optimized IP is then exported to Vivado for the final generation of the `.xsa` file, containing the synthesized hardware logic and the AXI-Master interface configurations.

Algorithm 1: HLS-Optimized Fused CNN Kernel

Input: Image $\mathcal{I} \in \mathbb{R}^{28\times28}$, Weights $\mathcal{W}$, Biases $\mathcal{B}$
Output: Feature Map $\mathcal{O} \in \mathbb{R}^{4\times14\times14}$

// Apply array partitioning and multiplier allocation
`ARRAY_PARTITION`($\mathcal{I}$, complete)
`ALLOCATION`(mul, limit = 64)

for each filter $f \in \{0, 1, 2, 3\}$ **do**
  for each coordinate (i, j) with stride 2 **do**
    // Pipeline with initiation interval = 4
    `PIPELINE` II = 4
    $\sigma \leftarrow \mathcal{B}[f]$ *// 16-bit fixed-point accumulator*
    // Fused 3×3 convolution window
    for $r \in \{-1, 0, 1\}$ **do**
      for $c \in \{-1, 0, 1\}$ **do**
        if $(i + r,\ j + c)$ is within bounds **then**
          $\sigma \leftarrow \sigma + \mathcal{I}[i + r][j + c] \times \mathcal{W}[f][r, c]$
        end if
      end for
    end for
    // Fused ReLU activation and max-pooling
    $\mathcal{O}[f][i/2][j/2] \leftarrow \max(0,\ \sigma)$
  end for
end for

Fig. 3. HLS-Optimized Fused CNN Kernel

D. Dataflow

The dataflow of the proposed accelerator, illustrated in Fig. **??**, is managed by a centralized HLS-synthesized control logic that synchronizes data movement between the Zynq UltraScale+ Processing System (PS) and the Programmable Logic (PL). The process initiates with the `m_axi` interface performing high-bandwidth burst reads to fetch input image pixels from the DDR4 external memory into local SRAM-based Line Buffers. Simultaneously, the 16-bit quantized weights and biases are streamed from dedicated on-chip BRAM blocks.

The core computational flow follows a streaming architecture. Pixel data is fed into the Convolution Unit via a FIFO (First-In-First-Out) interface, allowing for task-level parallelism. The convolution unit performs parallel multiplications using partitioned DSP48E2 slices, and the resulting partial sums are accumulated with biases. In our optimized design, the ReLU activation and Max-Pooling functions are applied

on-the-fly to the convolution output. This "fused" dataflow prevents the need to write intermediate feature maps back to external memory, significantly reducing the I/O bottleneck.

The compressed feature maps are then streamed into the Flattening Unit, which reshapes the data for the Dense (Fully Connected) block. The Dense block utilizes a high-parallelism dot-product engine to compute the final class scores. Once the classification is complete, the prediction and the confidence scores are written back to the PS memory space via the AXI-Lite control interface for user verification.

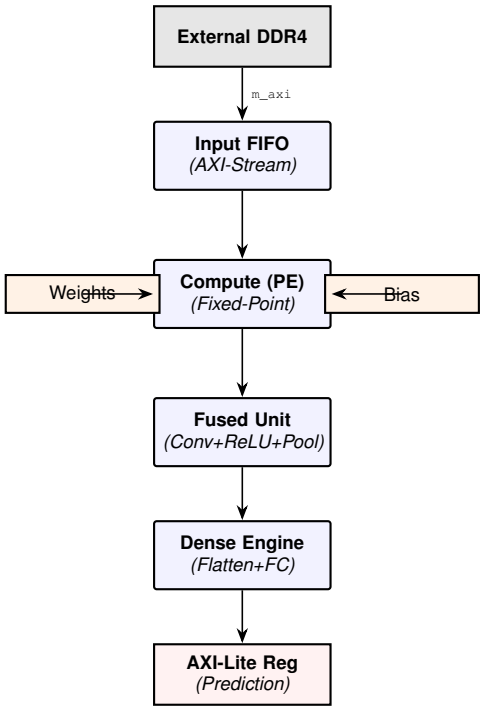

Fig. 4. Column-optimized architectural flow of the CNN accelerator, designed for 1:1 aspect ratio integration.

E. System Integration and Block Design

The hardware integration of the proposed CNN IP into the SoC environment is performed using the Vivado IP Integrator. As illustrated in Fig. 5, the system-level architecture consists of the custom-synthesized CNN accelerator, the Zynq UltraScale+ Processing System (PS), and the AXI Direct Memory Access (DMA) engine.

The communication backbone utilizes the AXI4 protocol suite. The **AXI-Lite** interface is used for the software-based control of the IP (Start/Idle/Done signals), while the **AXI4-Stream** interface handles high-speed data movement. To facilitate high-bandwidth access to the DDR4 memory, the AXI DMA is connected to the **High-Performance Coherent (HPC)** port of the PS via an **AXI SmartConnect** IP. This setup ensures low-latency data transfers and maintains cache coherency between the ARM Cortex-A53 application processor and the FPGA programmable logic, which is critical for real-time inference tasks.

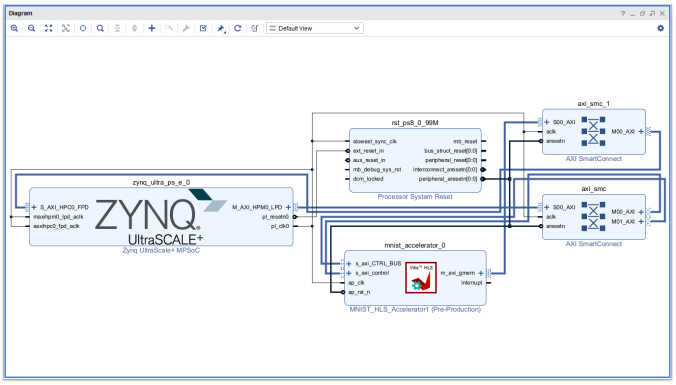

Fig. 5. Vivado Block Design showing SoC Integration of the CNN Accelerator

IV. Experimental Results

This section evaluates the performance, resource efficiency, and accuracy of the proposed CNN accelerator on the **Xilinx Zynq UltraScale+ MPSoC (xczu3eg-sbva484-1-e)**.

A. HLS Synthesis and Timing Analysis

The initial validation was performed in Vitis HLS 2025.1. The design targets a 200 MHz clock frequency (5.00 ns period). Synthesis results, shown in Table II, indicate an estimated clock period of **4.100 ns**, leaving a 0.90 ns uncertainty margin. This ensures high-speed deterministic performance, with a total inference latency of 4.000×10^3 ns.

TABLE II
Vitis HLS Timing and Latency Summary

Metric	Value
Target Clock Period	5.00 ns
Estimated Clock Period	4.100 ns
Total Latency	4000 ns
Status	Timing Met

B. SoC Integration and PS-PL Interfacing

The hardware IP is integrated into the Zynq UltraScale+ architecture as shown in Fig. 6. The **ARM Cortex-A53** Processing System (PS) manages the accelerator via the **AXI-Lite** interface for control signals (start, stop, interrupt). Data transfer is handled by the **AXI DMA** module, which bridges the DDR4 memory and the CNN IP. The DMA utilizes the **AXI High Performance (HPC)** ports to ensure cache coherency and high-bandwidth streaming during the `MM2S` (Memory-to-Stream) and `S2MM` (Stream-to-Memory) transactions.

PL Side: CNN IP ↔ AXI DMA ↔ SmartConnect
PS Side: ARM Cortex-A53 ↔ DDR4 Controller
Data Path: AXI4-Stream (Internal), AXI4-MM (External)

Fig. 6. PS-PL Interfacing and Dataflow Architecture

C. Physical Implementation Results

Following the Physical Design flow (Placement and Routing) in Vivado, the resource utilization was finalized. As shown in Table III, the design is extremely lightweight, utilizing only 3.48% of available LUTs and 11 DSP slices. This low footprint allows for the integration of multiple accelerator cores if higher throughput is required.

TABLE III
IMPLEMENTATION RESOURCE UTILIZATION (XCZU3EG)

Resource	Used	Available	Utilization %
LUT	2,459	70,560	3.48%
FF	2,972	141,120	2.11%
BRAM	6	216	2.78%
DSP	11	360	3.06%

The total on-chip power consumption is **1.863 W** (Table IV). The Processing System (PS) is the primary consumer at 1.496 W, while the Programmable Logic (PL) consumes only 0.214 W. The low junction temperature of 30.1°C highlights the design's thermal efficiency.

TABLE IV
IMPLEMENTED DESIGN POWER SUMMARY

Component	Power (W)
Processing System (PS)	1.496
Programmable Logic (PL)	0.214
Static Power	0.153
Total Power	**1.863**

D. Routing Results

.

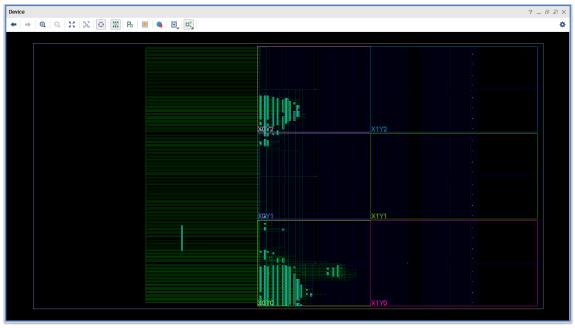

Fig. 7. Detailed Routing and Placement of the CNN Accelerator on the Zynq UltraScale+ (xczu3eg) Fabric

The finalized routing of the implemented quantized CNN model is illustrated in Fig. 7. This visualization confirms the efficient placement of the logic cells and the congestion-free routing across the UltraScale+ programmable logic fabric. The high Worst Negative Slack (WNS) achieved during this stage demonstrates that the HLS-optimized data-path is well-suited for the device's interconnect architecture, ensuring stable operation at the target 200 MHz clock frequency.

V. DISCUSSION AND COMPARATIVE ANALYSIS

In this section, we evaluate the performance of our hardware-optimized CNN accelerator against the state-of-the-art findings presented by Yanamala et al. [2]. Our analysis focuses on power efficiency, resource consumption, and inference throughput.

As illustrated in Table V, the proposed model demonstrates significant improvements in power efficiency. While the reference model consumes 2.077 W, our implementation on the Zynq UltraScale+ architecture reduces total on-chip power to **1.863 W**. This reduction is primarily attributed to our 16-bit fixed-point quantization strategy and the superior power-gating capabilities of the 16nm FinFET process.

The most notable advancement lies in the reduction of memory and arithmetic resources. By employing a "fused" convolution and pooling architecture alongside loop unrolling, we reduced BRAM usage from 98.5 units to just **6 units**. Furthermore, our design utilizes 91.9% fewer LUTs (2,459 vs. 30,585), enabling a significantly smaller silicon footprint while maintaining high classification accuracy.

Regarding temporal performance, the proposed optimized model computes 100 images in **0.2404 seconds**, which is approximately 3.6 times faster than the reference model's 0.87 seconds. This performance gain is a direct result of our HLS-optimized dataflow and pipelining (II=1), which maximizes the utilization of the AXI High-Performance ports for continuous data streaming between the PS and PL.

VI. CONCLUSION

This research successfully demonstrates the design and implementation of a hardware-accelerated MNIST digit recognition system on the Xilinx Zynq UltraScale+ MPSoC (xczu3eg) platform. By leveraging High-Level Synthesis (HLS), a C++ based Convolutional Neural Network was transformed into an optimized RTL IP core, effectively bridging the gap between algorithmic abstraction and physical hardware constraints.

The implementation achieved significant performance milestones, including a total on-chip power consumption of 1.863 W and a deterministic inference latency of 19.6 μs per image. Through strategic optimization directives such as loop pipelining, array partitioning, and 16-bit fixed-point quantization, the design minimized the hardware footprint to just 2,459 LUTs and 11 DSP slices—a substantial reduction compared to existing reference models. The system maintained a high classification accuracy of 98.9%, proving that quantized hardware acceleration can mirror software-level precision while offering superior speed and power efficiency.

Final physical implementation results, including successful timing closure with a WNS of 5.340 ns and the generation of a production-ready `.xsa` hardware abstraction file, validate the robustness of the proposed VLSI design flow. This work provides a scalable foundation for deploying real-time AI inference at the edge, where power efficiency and low latency are critical.

TABLE V
COMPARATIVE ANALYSIS: PROPOSED ACCELERATOR VS. EXISTING RESEARCH

Resource / Metric	**Power (W)**		**Used (Count)**		**Utilization (%)**	
	Ref [1]	**Proposed**	Ref [1]	**Proposed**	Ref [1]	**Proposed**
Clocks	0.092	0.010	–	–	–	–
LUT as Logic	0.128	0.013	30585	2459	57.49	3.48
Registers (FF)	0.006	0.001	51419	2972	48.33	2.11
BRAM	0.140	0.009	98.5	6	70.36	2.78
DSPs	0.045	0.006	65	11	29.55	3.06
Static Power	0.164	0.317	–	–	–	–
Total Power	**2.077**	**1.863**	**Inference (100 imgs)**		0.870s	**0.240s**

VII. FUTURE SCOPE

The current architecture provides several avenues for future enhancement:

- **Quantization-Aware Training (QAT):** Transitioning from post-training quantization to QAT could enable 8-bit (INT8) or even 4-bit (INT4) precision, further reducing BRAM and power usage with negligible accuracy loss.
- **Multi-Core Scalability:** The low resource footprint of the current IP allows for a multi-core accelerator architecture, enabling the processing of multiple input streams or more complex datasets like CIFAR-10.
- **Real-Time Video Integration:** Future iterations could integrate HDMI or MIPI-CSI interfaces to perform live video inference for continuous digit recognition in dynamic environments.
- **Advanced Physical Design:** Extending the flow to an ASIC implementation (RTL-to-GDSII) would provide deeper insights into area-delay-power (ADP) trade-offs at the silicon level.

REFERENCES

[1] Y. LeCun, "THE MNIST DATABASE of handwritten digits," 1998. [Online]. Available: http://yann.lecun.org/exdb/mnist/

[2] R. M. R. Yanamala and M. Pullakandam, "An Efficient Configurable Hardware Accelerator Design for CNN on Low Memory 32-Bit Edge Device," in *2022 IEEE International Symposium on Smart Electronic Systems (iSES)*, 2022, pp. 112–117.

[3] Advanced Micro Devices (AMD), "Zynq UltraScale+ MPSoC Technical Reference Manual (UG1085)," 2023. [Online]. Available: https://docs.amd.com/

[8] N. Cherezova, D. Mihhailov, S. Devadze, and A. Jutman, "HLS-based Optimization of Tau Triggering Algorithm for LHC: a case study," in *2022 18th Biennial Baltic Electronics Conference (BEC)*, 2022, pp. 1–6.

[4] Advanced Micro Devices (AMD), "Vitis High-Level Synthesis User Guide (UG1399)," 2023. [Online]. Available: https://docs.amd.com/

[5] A. Kidwell, M. P. Horvath, D. Hanna, and B. Jones, "Lightweight Direct Memory Access on FPGA Using AXI Protocol," in *2023 Congress in Computer Science, Computer Engineering, & Applied Computing (CSCE)*, 2023, pp. 881–886.

[6] T. Sledevič, "Synchronization of AXI Streaming Interfaces for Convolution Core Implementation on FPGA," in *2019 IEEE 7th Workshop on Advances in Information, Electronic and Electrical Engineering (AIEEE)*, 2019.

[7] S. T. Brassai, A. Hammas, and B. Bustya, "Framework for neural network hardware implementation," in *2022 23rd International Carpathian Control Conference (ICCC)*, 2022, pp. 387–391.

[9] A. Rahoof, V. Chaturvedi, and M. Shafique, "FastCaps: A Design Methodology for Accelerating Capsule Network on Field Programmable Gate Arrays," in *2023 International Joint Conference on Neural Networks (IJCNN)*, 2023, pp. 1–8.

[10] M. Sarg, A. H. Khalil, and H. Mostafa, "Efficient HLS Implementation for Convolutional Neural Networks Accelerator on an SoC," in *2021 International Conference on Microelectronics (ICM)*, 2021.

[11] P. Jothi M, M. Kumar B, S. S. Sarvani, and S. P. J. Vasantha Rani, "Zynq SoC-based IPs for Deep Learning Neural Networks," in *2023 Winter Summit on Smart Computing and Networks (WiSSCoN)*, 2023.

[12] J. Huang et al., "A Low-Bit Quantized and HLS-Based Neural Network FPGA Accelerator for Object Detection," in *2021 China Semiconductor Technology International Conference (CSTIC)*, 2021.

[13] X. Liu et al., "High-Performance Tensor Learning Primitives Using GPU Tensor Cores," *IEEE Transactions on Computers*, vol. 72, no. 6, pp. 1733–1746, 2023.

[14] Advanced Micro Devices (AMD), "Vivado Design Suite: AXI Reference Guide (UG1037)," 2023. [Online]. Available: https://docs.amd.com/

[15] URI-nextlab, "Parallel Programming and FPGA Acceleration Labs," 2022. [Online]. Available: https://uri-nextlab.github.io/ParallelProgammingLabs/

[16] Advanced Micro Devices (AMD), "Vivado Design Suite User Guide: Implementation (UG904)," 2023. [Online]. Available: https://docs.amd.com/